

- Content-Security-Policy (CSP) на платформе KOTOKOD — полный разбор
 - Оглавление
 - Часть 0. Кому и зачем этот документ
 - Часть 1. Фундамент: модель безопасности браузера
 - 1.1. Что такое «origin» (источник)
 - 1.2. Same-Origin Policy (SOP) — правило по умолчанию
 - 1.3. Почему SOP недостаточно — и тут выходит CSP
 - Часть 2. Две угрозы: XSS и CSRF
 - 2.1. Три вида XSS (важно для понимания, что ловит CSP)
 - 2.2. Ключевая мысль: XSS сильнее CSRF
 - Часть 3. Почему CSP нужен именно СЕЙЧАС (сдвиг модели угроз)
 - 3.1. Было (Express, HMAC-cookie)
 - 3.2. Стало (Django, JWT-cookie + double-submit CSRF)
 - 3.3. Следствие и связка, которую надо запомнить
 - 3.4. Почему это defense-in-depth, а не «горящая дыра»
 - Часть 4. CSP как механизм: теория директив и источников
 - 4.1. Что это технически
 - 4.2. Анатомия: директивы и списки источников
 - 4.3. Категории директив
 - 4.4. Список источников: что можно писать
 - 4.5. Как браузер это применяет (пошагово)
 - 4.6. Почему script-src 'self' рубит XSS
 - Часть 5. Наша политика — директива за директивой
 - Часть 6. Главный блокер: рефактор teacher SPA
 - 6.1. Инвентаризация (факты по коду)
 - 6.2. Что было (пример до)
 - 6.3. Что стало (после)
 - 6.4. Три приёма привязки (и когда какой)
 - 6.5. Почему именно внешний файл, а не nonce/hash/unsafe
 - 6.6. Тайминг (почему не сломалось)
 - Часть 7. Где живёт CSP: nginx, а не Django
 - Часть 8. Сбор нарушений: отчёты и наш nginx-механизм
 - 8.1. Что браузер делает при нарушении
 - 8.2. Схема отчёта (что внутри JSON)
 - 8.3. report-uri против report-to (теория)
 - 8.4. Наш механизм: nginx без Django

- 8.5. Приятный бонус: footgun «ложно-зелёного» обойдён по построению
- 8.6. Формат лога и разбор
- Часть 9. Грабли nginx: наследование `add_header`
- Часть 10. Report-Only против enforce: семантика и различия
 - 10.1. Чем отличаются
 - 10.2. Важно: не все директивы работают в Report-Only
 - 10.3. Зачем вообще фаза Report-Only
- Часть 11. Практические плейбуки
 - 11.1. Запустить и проверить локально
 - 11.2. Проверить, что заголовок реально отдаётся
 - 11.3. Разобрать лог нарушений (jq-рецепты)
 - 11.4. Я вижу нарушение — как чинить (алгоритм)
 - 11.5. Worked example: добавить новый разрешённый источник
 - 11.6. Worked example (backlog): self-host Google Fonts → ужесточить политику
- Часть 12. CSP в связке с другими security-заголовками
- Часть 13. Решения, которые НЕ меняем, и backlog
 - Сознательно оставляем
 - Backlog (не блокирует включение CSP)
- Часть 14. FAQ и типичные ошибки
- Часть 15. Глоссарий
- Часть 16. Шпаргалка + где это в коде
 - Шпаргалка
 - Где это в коде

Content-Security-Policy (CSP) на платформе KOTOKOD — полный разбор

Обучающий материал: и теория «с нуля», и практика конкретно нашего проекта. Читается сверху вниз; первые части — общая теория браузерной безопасности и CSP, дальше — как это устроено у нас (с реальными фрагментами кода), плейбуки на каждый день и FAQ. Парный существующий документ: [docs/accounts-explained.md](#) (кто пользователь и что ему можно). Документ [docs/auth-explained.md](#) (cookie/JWT/CSRF с

азов) пока **не написан** — планируется как companion; модель угроз CSRF кратко дана здесь в Части 3.

Статус реализации (2026-06-19): включён режим **Report-Only** — браузер логирует нарушения, но ничего не блокирует. Боевой **Content-Security-Policy** ещё НЕ включён (ждёт браузерного прогона трёх фронтов).
Подробнее — Часть 11.

Оглавление

- Часть 0. Кому и зачем этот документ
- Часть 1. Фундамент: модель безопасности браузера (Origin, Same-Origin Policy)
- Часть 2. Две угрозы: XSS и CSRF (и почему XSS сильнее)
- Часть 3. Почему CSP нужен именно сейчас (сдвиг модели угроз в нашем проекте)
- Часть 4. CSP как механизм: теория директив и источников
- Часть 5. Наша политика — директива за директивой
- Часть 6. Главный блокер: рефактор teacher SPA (с кодом)
- Часть 7. Где живёт CSP: nginx, а не Django
- Часть 8. Сбор нарушений: отчёты, report-uri/report-to, наш nginx-механизм
- Часть 9. Грабли nginx: наследование **add_header**
- Часть 10. Report-Only против enforce: семантика и различия
- Часть 11. Практические плейбуки (запуск, отладка, добавление источника)
- Часть 12. CSP в связке с другими security-заголовками
- Часть 13. Решения, которые НЕ меняем, и backlog
- Часть 14. FAQ и типичные ошибки
- Часть 15. Глоссарий
- Часть 16. Шпаргалка + «где это в коде»

Часть 0. Кому и зачем этот документ

Цель — чтобы человек, **не знавший про CSP вообще**, после прочтения мог: понять, от чего CSP защищает; прочитать нашу политику и объяснить каждую

строчку; найти и починить нарушение; безопасно добавить новый источник (CDN, шрифт, API); и включить боевой режим.

Если нужно быстро — читайте Часть 16 (шпаргалка). Если нужно глубоко — по порядку.

Часть 1. Фундамент: модель безопасности браузера

1.1. Что такое «origin» (источник)

Origin — это тройка **схема + хост + порт**. Например:

```
https://kotokod.ru:443    ← origin A
http://kotokod.ru:443     ← другой origin (схема http ≠ https)
https://api.kotokod.ru:443 ← другой origin (хост api. ≠ голый)
https://kotokod.ru:8443   ← другой origin (порт 8443 ≠ 443)
```

Меняется хоть один из трёх — это **другой** origin. Это базовая единица изоляции в вебе.

1.2. Same-Origin Policy (SOP) — правило по умолчанию

Same-Origin Policy — фундаментальное правило браузера: код одной страницы по умолчанию **не может читать данные** из другого origin. Скрипт с **evil.com** не прочитает ответ `fetch('https://kotokod.ru/api/me')` — браузер не отдаст ему тело. Именно SOP не даёт чужим сайтам воровать ваши данные «через JS».

1.3. Почему SOP недостаточно — и тут выходит CSP

SOP защищает **между** origin. Но если зловредный код **исполнился внутри** **нашего же origin** (а именно это делает XSS — см. Часть 2), для SOP он «свой», и

SOP его не остановит. Нужен механизм, который ограничивает **что вообще может исполняться и грузиться на нашей странице** — даже будучи «своим». Это и есть **CSP**.

Аналогия: SOP — это забор между домами (чужой не зайдёт в твой двор). CSP — это правила внутри дома: «исполнять можно только скрипты из вот этого сейфа, чужие бумажки на полу не читаем, как бы они туда ни попали».

Часть 2. Две угрозы: XSS и CSRF

	XSS (Cross-Site Scripting)	CSRF (Cross-Site Request Forgery)
Суть	Внедрить свой JS в нашу страницу, и он исполнится в браузере жертвы	С чужого сайта заставить браузер жертвы послать запрос на наш сервер от её имени
Где живёт зловред	Внутри нашего origin (kotokod.ru)	На постороннем сайте (evil.com)
Что может	Читать DOM, cookie (не-HttpOnly), localStorage, csrf-токен; слать запросы; красть данные	Только «вслепую» вызвать наш мутирующий эндпоинт (ответа не видит — мешает SOP)
Защита	CSP + экранирование вывода + HttpOnly-cookie	CSRF-токен (double-submit), SameSite-cookie

2.1. Три вида XSS (важно для понимания, что ловит CSP)

- **Reflected (отражённый)**: зловредный скрипт прилетает в URL/парамetre и сервер «отражает» его в HTML без экранирования. Пример: `?q=<script>steal()</script>`.
- **Stored (хранимый)**: скрипт сохраняется в БД (коммент, имя) и показывается другим пользователям. Самый опасный — бьёт по всем, кто открыл

страницу.

- **DOM-based:** сервер ни при чём, уязвимость в самом клиентском JS, который берёт данные из URL/`location` и пихает в `innerHTML/eval` без очистки.

CSP с `script-src 'self'` (без `'unsafe-inline'`) рубит **reflected** и **stored** почти полностью: внедрённый `<script>...</script>` или `<img onerror=...>` — это inline-код, его браузер не исполнит. DOM-based частично: `eval/new Function` запрещены (`'unsafe-eval'` не задан), но `element.innerHTML = userInput` со ссылкой на внешний скрипт всё равно упрётся в `script-src`. CSP — **второй рубеж** (defense-in-depth), не замена экранированию.

2.2. Ключевая мысль: XSS сильнее CSRF

Если на нашей странице исполнился чужой скрипт (XSS), он работает «изнутри» и видит всё, что видит наш собственный JS, — **включая csrf-токен**. Значит он может подделать любой мутирующий запрос. **XSS обходит CSRF-защиту**. Обратное неверно: CSRF не даёт исполнять скрипты. Вывод: **XSS — корневая угроза, а главный барьер против неё — CSP**.

Часть 3. Почему CSP нужен именно СЕЙЧАС (сдвиг модели угроз)

CSP был полезен всегда, но в нашем проекте недавно **выросла цена** его отсутствия.

3.1. Было (Express, HMAC-cookie)

CSRF-токена в JavaScript **не существовало**. Сессия — HttpOnly-cookie, JS до неё не дотягивается. XSS мог напасть, но «ключ от сессии» не крал.

3.2. Стало (Django, JWT-cookie + double-submit CSRF)

Паттерн double-submit требует, чтобы SPA **читала** csrf-токен из `document.cookie` и клала его в заголовок `X-CSRFToken`. Поэтому в настройках:

```
# production.py / base.py
CSRF_COOKIE_HTTPONLY = False # ← csrf-cookie ДОЛЖНА быть видна из JS (требование double-submit)
```

Это **требование** механизма, а не недосмотр. Но следствие: csrf-токен теперь доступен из JavaScript.

3.3. Следствие и связка, которую надо запомнить

Раз csrf-токен читается из JS — любой исполнившийся чужой скрипт (XSS) его прочитает и подделает запрос. **CSRF-защита больше не прикрывает XSS.** Единственная оставшаяся мера defense-in-depth против XSS — **CSP**, которого не было.

```
CSRF_COOKIE_HTTPONLY=False (требование double-submit)
  → csrf-токен читается из JS
  → XSS, если случится, обходит CSRF
  → нужен CSP, чтобы не дать XSS исполниться вообще
```

3.4. Почему это defense-in-depth, а не «горящая дыра»

Сами JWT `access/refresh`-cookie остаются `HttpOnly=True` — украсть саму сессию через XSS нельзя, можно лишь «прокатиться» на ней, пока открыта вкладка. CSP отсутствовал и раньше; переход на JWT лишь повысил цену. Поэтому — плановая мини-фаза, не hotfix.

Часть 4. CSP как механизм: теория директив и источников

4.1. Что это технически

CSP — это **HTTP-заголовок ответа**. Сервер в нём перечисляет, откуда страницы можно грузить и что исполнять. **Принуждает браузер**, не сервер. Есть два заголовка:

Заголовок	Что делает
<code>Content-Security-Policy</code>	Боевой : нарушения блокируются И логируются
<code>Content-Security-Policy-Report-Only</code>	Тест : нарушения только логируются, ничего не блокируется

Немного истории: CSP 1 (2012) — базовые `*-src`; CSP 2 (2015) — `nonce`, `hash`, `frame-ancestors`, `form-action`; CSP 3 (черновик, широко поддержан) — `strict-dynamic`, `'unsafe-hashes'`, Reporting API. Мы используем подмножество, поддержанное всеми браузерами.

4.2. Анатомия: директивы и списки источников

Политика — это `;`-разделённый список **директив**, у каждой — **список источников**:

```
script-src 'self' https://cdn.example.com;  
└─ директива ─┬── список источников ───┘
```

4.3. Категории директив

- **Fetch-директивы** — откуда грузить ресурсы данного типа: `script-src`, `style-src`, `img-src`, `font-src`, `connect-src` (fetch/XHR/WebSocket/EventSource), `media-src` (audio/video), `object-src` (`<object>/<embed>`), `frame-src` (что Я встраиваю в `iframe`), `worker-src`, `manifest-src`, `child-src`.
- **Документные директивы**: `base-uri` (ограничивает `<base href>`), `sandbox` (как у `iframe-sandbox`, но для всей страницы).

- **Навигационные:** `form-action` (куда формы могут сабмититься), `frame-ancestors` (кто может встроить НАС в `iframe` — обратное к `frame-src`), `navigate-to`.
- **Reporting:** `report-uri` (старый), `report-to` (новый, через Reporting-API).
- **Прочие/поведенческие:** `upgrade-insecure-requests` (апгрейд `http://` → `https://`), `block-all-mixed-content` (deprecated).
- **Фолбэк:** `default-src` — значение по умолчанию для всех **fetch**-директив, которые не заданы явно. Навигационные/документные он НЕ покрывает (их задают отдельно).

4.4. Список источников: что можно писать

Запись	Значение
'self'	Тот же origin (схема+хост+порт). НЕ включает <code>data:/blob:/</code> субдомены
'none'	Пусто — запретить всё для этой директивы
<code>https://fonts.gstatic.com</code>	Конкретный хост (host-source). Можно с путём, портом
<code>https:</code>	Любой хост по этой схеме (scheme-source)
<code>data: blob:</code>	Разрешить data-URI / blob-URI
<code>*.googleapis.com</code>	Wildcard поддомена
'unsafe-inline'	Разрешить inline (<code><script>...</script></code> , <code>onclick=</code> , <code>style=</code>). Для <code>script-src</code> — опасно
'unsafe-eval'	Разрешить <code>eval/new Function</code> /строковый <code>setTimeout</code>
'nonce-XYZ'	Разрешить inline-тег с атрибутом <code>nonce="XYZ"</code> (токен — на каждый запрос новый)
'sha256-...'	Разрешить конкретный inline-блок по хешу его содержимого
'strict-dynamic'	Доверять скриптам, загруженным уже доверенным скриптом (CSP3)

Запись	Значение
'unsafe-hashes'	Разрешить inline-обработчики (onclick=) по хешу (узкий случай)

4.5. Как браузер это применяет (пошагово)

1. Получив ответ, браузер парсит заголовок CSP в набор правил.
2. На **каждую** попытку загрузить/исполнить ресурс (скрипт, стиль, картинку, fetch, iframe...) браузер находит соответствующую директиву (или default-src) и проверяет, матчится ли источник.
3. **Матч** → разрешить. **Нет матча** → в боевом режиме **заблокировать** + сгенерировать нарушение; в Report-Only — только сгенерировать нарушение.
4. Нарушение пишется в консоль DevTools и (если задан report-uri/report-to) POST'ится на endpoint отчётов.

4.6. Почему script-src 'self' рубит XSS

'self' = «только файл с нашего origin». При нём браузер **откажется** исполнять:

- `<script>...inline...</script>` — inline-блок (типичный вектор stored/reflected XSS);
- `<button onclick="evil()">` — inline-обработчик (тот же inline-JS в атрибуте);
- `<img src=x onerror="evil()">` — inline-обработчик через событие;
- `eval("..."), new Function("..."), setTimeout("строка", ...)` — исполнение строк;
- `<script src="https://evil.com/x.js">` — скрипт с чужого origin.

⚠️ Добавить 'unsafe-inline' в script-src — значит снова разрешить всё это и **обнулить защиту от XSS**. Поэтому у нас 'unsafe-inline' в script-src под запретом. (В style-src он допустим — риск стилей несопоставимо ниже, см. Часть 5.)

Часть 5. Наша политика — директива за директивой

Точное значение (идентично в прод- и локальном конфиге; одна строка в `add_header`):

```
default-src 'self';
script-src 'self';
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
img-src 'self' data:;
font-src 'self' https://fonts.gstatic.com;
connect-src 'self';
frame-ancestors 'none';
base-uri 'self';
form-action 'self';
object-src 'none';
upgrade-insecure-requests;
report-uri /csp-report
```

Директива	Почему так / что сломается без неё
<code>default-src 'self'</code>	Фолбэк для незадаанных fetch-директив (<code>media-src</code> , <code>worker-src</code> , <code>manifest-src</code> , <code>frame-src</code> ...). Если что-то полезет из неожиданного места — упрётся в <code>'self'</code> и попадёт в отчёт.
<code>script-src 'self'</code>	Ядро. Без <code>'unsafe-inline'/'unsafe-eval'</code> . Возможно только после выноса inline-JS teacher SPA (Часть 6).
<code>style-src 'self' 'unsafe-inline' https://fonts.googleapis.com</code>	<code>'unsafe-inline'</code> — осознанный компромисс: React генерит <code>style={{{}}</code> рантайм (хешировать нечего, попсе негде взять), teacher имеет <code>style="..."</code> . Кража токена через CSS невозможна. <code>fonts.googleapis.com</code> — Google Fonts CSS у teacher и admin.
<code>img-src 'self' data:</code>	Свои картинки + <code>data:-URI</code> (inline-иконки/SVG).

Директива	Почему так / что сломается без неё
<code>font-src 'self'</code> <code>https://fonts.gstatic.com</code>	Локальные шрифты (Steppe в <code>/fonts/</code>) + файлы Google Fonts (их отдаёт <code>gstatic</code>).
<code>connect-src 'self'</code>	Все <code>fetch</code> идут на относительные <code>/api/*</code> (same-origin). Закрывает канал эксфильтрации данных на чужой сервер.
<code>frame-ancestors 'none'</code>	Нас никто не встроит в iframe (anti-clickjacking). Усиливает <code>X-Frame-Options</code> .
<code>base-uri 'self'</code>	Запрет подмены <code><base href></code> (иначе относительные URL ресурсов можно увести на чужой хост).
<code>form-action 'self'</code>	Формы (login) сабмитятся только на свой origin.
<code>object-src 'none'</code>	Никаких <code><object>/<embed></code> — legacy-плагины (Flash и пр.) исторически дырявы.
<code>upgrade-insecure-requests</code>	Браузер апгрейдит случайный <code>http://</code> -подресурс до <code>https://</code> (страховка от mixed-content). Внимание: в Report-Only НЕ действует (Часть 10).
<code>report-uri /csp-report</code>	Куда слать отчёты о нарушениях (Часть 8).

Остаточный риск, который `script-src 'self'` НЕ закрывает: загрузка скрипта с **нашего же** origin. Сейчас таких эндпоинтов нет. Когда появятся user-uploads/файлы в кабинетах — `script-src 'self'` надо пересмотреть (раздавать загруженное с отдельного origin или с `Content-Disposition: attachment`).

Часть 6. Главный блокер: рефактор teacher SPA

6.1. Инвентаризация (факты по коду)

Чтобы поставить честный `script-src 'self'`, на страницах не должно остаться inline-JS.

Фронт	inline <code><script></code>	inline <code>on*=</code>	вердикт
<code>login/index.html</code> , <code>login/set-password.html</code>	0 (внешний <code>login.js/set-password.js</code>)	0	чисто
<code>admin-dist/index.html</code>	0 (внешний Vite-бандл <code>type="module"</code>)	0	чисто
<code>teacher/index.html</code>	1 (1364 строки)	27 (23 <code>onclick</code> , 3 <code>onchange</code> , 1 <code>oninput</code>)	блокер

`login` и `admin` были совместимы изначально. Весь блокер — `teacher SPA` (vanilla JS).

6.2. Что было (пример до)

```
<!-- index.html: inline-скрипт + обработчики прямо в разметке -->
<button id="submitBtn" onclick="submitForm()" disabled>Сохранить урок</button>
<button class="rf-chip" data-status="done"
onclick="setReportStatusFilter('done')">...</button>
...
<script>
  "use strict";
  function submitForm() { /* 1364 строки логики */ }
  ...
  // даже в template-литерале попался inline-обработчик:
  html += `<button class="popup-close" onclick="hideSchedPopup()">X</button>`;
</script>
```

Под `script-src 'self'` НЕ работало бы ничего из этого: ни `inline-<script>`, ни 27 `on*=`, ни 28-й `onclick`, спрятанный в строке шаблона попапа (`innerHTML`).

6.3. Что стало (после)

1. Весь inline-`<script>` (1364 строки) вынесен в `frontend/teacher/app.js`, подключён как `<script src="/teacher/app.js"></script>`. Это **классический** скрипт (НЕ `type="module"`): код опирается на глобальные функции и `window.onload`; модуль изменил бы область видимости и тайминг.
2. Все 27 `on*=` убраны из HTML; 28-й — из template-строки. Поведение — без изменений.
3. Привязка обработчиков — в функции `wireStaticHandlers()`:

```
// frontend/teacher/app.js — реальный код
function wireStaticHandlers() {
  const on = (id, ev, fn) => { const el = $(id); if (el) el.addEventListener(ev, fn); };

  // Верхняя навигация / журнал — по id
  on("btnJournal", "click", () => switchPage("journal"));
  on("submitBtn", "click", () => submitForm());
  on("groupSelect", "change", () => onGroupChange());
  on("recordUrl", "input", () => onRecordInput());
  // ...(всего 14 одиночных по id)...

  // Группа фильтров отчёта — ОДИН слушатель, делегирование по data-status
  const statusRow = $("rfStatusRow");
  if (statusRow) statusRow.addEventListener("click", e => {
    const chip = e.target.closest(".rf-chip");
    if (chip && chip.dataset.status) setReportStatusFilter(chip.dataset.status);
  });

  // <select> — значение берём из e.target (раньше было this.value)
  on("rfTeacherSelect", "change", e => setReportTeacherFilter(e.target.value));

  // Переключатель вида расписания — делегирование по data-view
  const viewTabs = document.querySelector(".view-tabs-sched");
  if (viewTabs) viewTabs.addEventListener("click", e => {
    const tab = e.target.closest(".view-tab-sched");
    if (tab && tab.dataset.view) setSchedView(tab.dataset.view);
  });

  // Попап: клик по фону закрывает; X создаётся динамически → тоже делегирование
  const overlay = $("schedOverlay");
  if (overlay) overlay.addEventListener("click", e => {
    if (e.target === e.currentTarget) { hideSchedPopup(); return; }
    if (e.target.closest(".popup-close")) hideSchedPopup();
  });
}

// Навешиваем, как только DOM готов (а если уже готов — сразу):
if (document.readyState === "loading") {
  document.addEventListener("DOMContentLoaded", wireStaticHandlers);
} else {
  wireStaticHandlers();
}
```

6.4. Три приёма привязки (и когда какой)

- По **id** — для уникальных элементов: `on("submitBtn","click",...)`. Просто и явно.
- Делегирование по **data-*/классу** — для групп однотипных кнопок и для элементов, которые **создаются динамически** (`innerHTML`). Один слушатель на родителя ловит клики по всем детям через `event.target.closest('.класс')`. Так сделаны: 5 фильтров `.rf-chip`, 3 вкладки `.view-tab-sched`, кнопка `.popup-close` внутри `#schedOverlay`.
- Свойство **DOM** (`element.onclick = () => ...`) — для динамических строк/карточек **внутри самого скрипта** (студенты, карточки уроков). Это присваивание свойства, **не inline-HTML**, и `script-src 'self'` его **разрешает** — поэтому такие места НЕ трогали.

6.5. Почему именно внешний файл, а не nonce/hash/unsafe

Вариант	Вердикт
<code>'unsafe-inline'</code> в <code>script-src</code>	Обнуляет защиту от XSS. Нет.
<code>hash ('sha256-...')</code>	Хеш пересчитывать при каждой правке; на 27 обработчиков — кошмар поддержки.
<code>nonce ('nonce-...')</code>	Нужен per-request токен при рендере шаблона. Статику отдаёт nginx без рендера — инжектировать негде.
внешний файл + <code>addEventListener</code> ✓	Покрывается <code>'self'</code> бесплатно, переживает любые правки, нативный DOM API. Канон для vanilla-JS.

6.6. Тайминг (почему не сломалось)

`wireStaticHandlers` навешивается на `DOMContentLoaded` (или сразу, если DOM уже готов) — **строго раньше** `window.onload`, который лишь запускает загрузку

данных и интервалы. Все привязываемые элементы статически есть в `index.html`, поэтому к моменту привязки они уже в DOM. Гонок нет.

Часть 7. Где живёт CSP: nginx, а не Django

CSP-заголовок (как и остальные security-заголовки) задаётся в **nginx**, на **server-уровне**, в обоих конфигах. Не в Django. Почему так — осознанное архитектурное решение:

1. **Статику отдаёт nginx напрямую.** `login`, `teacher`, `admin`-сборка — это файлы, которые nginx раздаёт без участия Django (нет рендера шаблона). Значит `django-csp` middleware физически не в потоке этих ответов, а per-request **none инжектить негде** (для none нужен серверный рендер HTML).
2. **Единый источник истины.** Все security-заголовки уже живут в nginx (helmet-эквивалент). Раздвоить их («заголовки в nginx, CSP в Django») = два места для аудита и риск рассинхрона.
3. **dev/prod parity.** Локальный nginx (`deploy/nginx/local/nginx.conf`) зеркалит прод — политика байт-в-байт одинаковая, тестируем ровно то, что поедет в прод.

Реальный блок (прод, `deploy/nginx/journal-kotokod.conf`):

```
# ===== Content-Security-Policy (defense-in-depth против XSS) =====
# ФАЗА ВЫКАТКИ: Report-Only — браузер ЛОГИРУЕТ нарушения, но НИЧЕГО не блокирует.
add_header Content-Security-Policy-Report-Only "default-src 'self'; script-src
'self'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; img-src
'self' data:; font-src 'self' https://fonts.gstatic.com; connect-src 'self'; frame-
ancestors 'none'; base-uri 'self'; form-action 'self'; object-src 'none'; upgrade-
insecure-requests; report-uri /csp-report" always;
```

`always` — слать заголовки даже на ответах-ошибках (4xx/5xx). Без него CSP не попал бы, например, на страницу 404.

`django-csp` оправдан там, где Django CAM рендерит HTML и нужен per-view none. У нас такого слоя нет — поэтому nginx.

Часть 8. Сбор нарушений: отчёты и наш nginx-механизм

8.1. Что браузер делает при нарушении

1. Пишет ошибку в консоль DevTools.
2. Если задан `report-uri/report-to` — POST'ит JSON-отчёт о нарушении.

8.2. Схема отчёта (что внутри JSON)

Тело отчёта `report-uri` выглядит так (поля по спецификации):

```
{
  "csp-report": {
    "document-uri": "https://kotokod.ru/teacher", // где случилось
    "referrer": "",
    "violated-directive": "script-src",           // какая директива нарушена
    "effective-directive": "script-src",          // фактически применённая (с
учётom фолбэка)
    "original-policy": "default-src 'self'; ...", // вся политика целиком
    "disposition": "report",                     // report (RO) | enforce
(боевой)
    "blocked-uri": "inline",                     // ЧТО заблокировано (URL |
"inline" | "eval")
    "status-code": 200,
    "script-sample": "...",                      // кусочек кода-нарушителя
(если доступен)
  }
}
```

Для тюнинга политики важнее всего `effective-directive` (что чинить) и `blocked-uri` (кого разрешить/убрать).

8.3. `report-uri` против `report-to` (теория)

- `report-uri <url>` — старый способ (CSP2). Браузер POST'ит `application/csp-report` на URL. Помечен deprecated, но **поддерживается всеми браузерами** и работает долго.

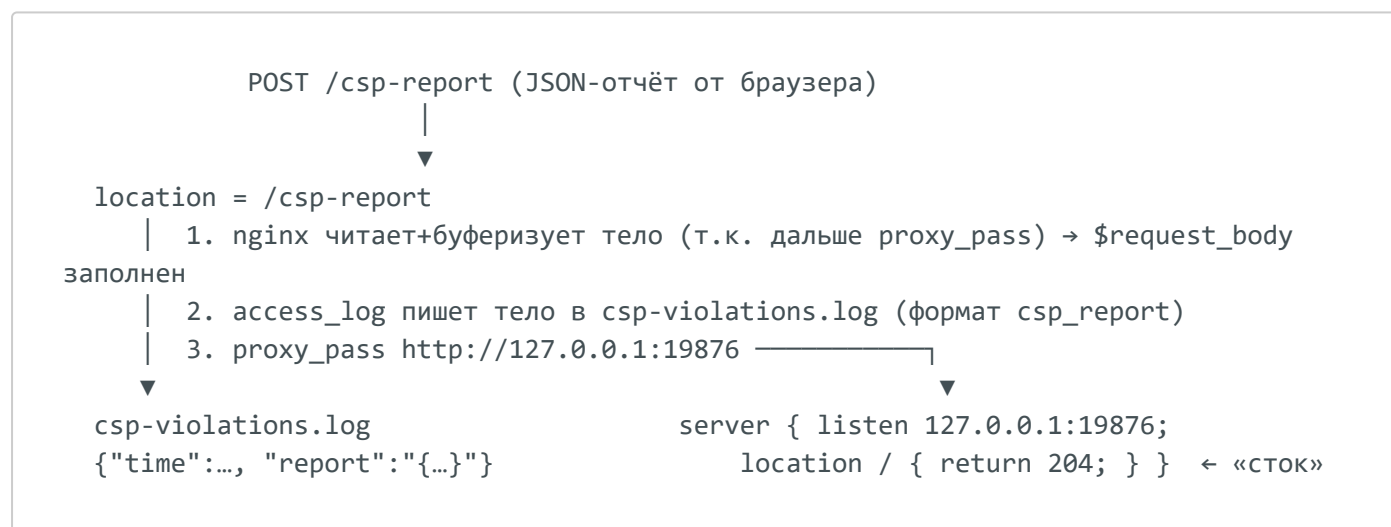
- **report-to <group> + Reporting-Endpoints/Report-To** — новый Reporting-API (CSP3). Гибче (батчинг, ретраи, другие типы репортов), но сложнее настраивать и местами поддержан неравномерно.
- **Наш выбор:** **report-uri** для старта (проще, работает везде). Переход на **report-to** (или оба сразу как фолбэк) — в backlog.

8.4. Наш механизм: nginx без Django

Требование было — **не плодить эндпоинт на Django** (нулевой код в приложении, pytest не трогаем). Значит отчёты принимает сам nginx. Тут тонкость, которую важно понять.

Тонкость: логировать нужно **тело** отчёта (там **blocked-uri/effective-directive**). Но переменная **\$request_body** в nginx **заполняется только если тело прочитано и буферизовано**, а это происходит лишь при **proxy_pass**. Если ответить **return 204**, nginx тело не читает → в лог попадёт пустота.

Решение — self-proxy на no-op «сток»:



Реальные блоки (прод, http-контекст):

```

# rate-limit зона: /csp-report – публичный write-в-лог, лимитируем от флуда
limit_req_zone $binary_remote_addr zone=api_csp:5m rate=30r/m;

# $request_body в кавычках: escape=json экранирует его как JSON-строку →
# строка лога остаётся валидным JSON, отчёт достаётся `jq '.report | fromjson'`.
log_format csp_report escape=json

'{"time":"$time_iso8601","ip":"$remote_addr","ref":"$http_referer","ua":"$http_user_agent","report":"$request_body"}';

# No-op сток на loopback: принимает проксированный отчёт и сразу 204.
  
```

```
server {
    listen 127.0.0.1:19876;
    server_name _;
    access_log off;
    location / { return 204; }
}
```

```
# внутри server{} (443): сам приёмник отчётов
location = /csp-report {
    limit_req zone=api_csp burst=10 nodelay; # анти-флуд (только прод)
    limit_req_status 429;
    access_log /var/log/nginx/csp-violations.log csp_report;
    proxy_pass http://127.0.0.1:19876; # ← вынуждает прочитать тело
    proxy_set_header Host $host;
    client_body_buffer_size 64k; # = max → отчёт всегда буферизуется
    в память
    client_max_body_size 64k; # ограничивает DoS-вектор
}
```

Локальный конфиг — то же самое, но: лог в `logs/csp-violations.log` (относительный путь под `nginx`-префиксом `-p`) и **без** `limit_req` — на dev это один человек, а лимит во время Report-Only-прогона глушил бы те самые отчёты, которые мы собираем.

8.5. Приятный бонус: footgun «ложно-зелёного» обойдён по построению

Частая ошибка таких схем: endpoint отчётов случайно попадает под CSRF/auth-middleware, все отчёты получают **403**, и кажется «нарушений нет», хотя их просто никто не принял. У нас это невозможно: `/csp-report` отвечает **204** из `nginx`-стока и **не касается Django**. (Проверено живым POST → **204**, тело в логе — валидный JSON.)

8.6. Формат лога и разбор

Одна строка = один отчёт = валидный JSON; сам отчёт лежит в `report` как JSON-строка:

```
{"time":"2026-06-19T14:19:56+03:00","ip":"127.0.0.1","ref":"","ua":"...","report":
{"csp-report":{"violated-directive":"script-src","blocked-
```

```
uri\":"inline\"}"} }
```

Рецепты `jq` (см. Часть 11.3).

Часть 9. Грабли nginx: наследование `add_header`

CSP-заголовков на **server-уровне** должен наследоваться во все `location`, включая раздачу статики из `snippets/journal-static.conf`. Но в nginx правило коварное:

`add_header` с **server-уровня** наследуется в `location` **ТОЛЬКО** пока этот `location` не объявляет **СВОЙ** `add_header`. Появился хоть один `add_header` внутри `location` — и **все** server-уровневые заголовки в нём **молча исчезают** (CSP, nosniff, X-Frame-Options...).

Поэтому у нас:

- Ни один `static-location` в сниппете, ни `/api/*`, ни `/csp-report` **не объявляют своих** `add_header` → они наследуют CSP и прочие security-заголовки.
- Проверено: `curl -I` / браузер на `/`, `/teacher`, `/admin` отдают `Content-Security-Policy-Report-Only`.

Если когда-нибудь понадобится `add_header` внутри `location` — придётся продублировать там **ВСЕ** server-уровневые заголовки, иначе они пропадут именно в нём.

Часть 10. Report-Only против enforce: семантика и различия

10.1. Чем отличаются

- `Content-Security-Policy-Report-Only` — нарушения **логируются**, загрузка **не блокируется**. Поле `disposition` в отчёте = `"report"`.
- `Content-Security-Policy` — нарушения **блокируются** И логируются. `disposition` = `"enforce"`.

10.2. Важно: не все директивы работают в Report-Only

Report-Only по определению «ничего не меняет», поэтому директивы, которые **меняют поведение** (а не просто отчитываются), в нём **игнорируются**:

- `upgrade-insecure-requests` — **не действует** в RO (апгрейд `http`→`https` поменял бы поведение). Заработает только в боевом режиме. У нас он в политике уже сейчас — это «заряд на будущее», но пока эффекта не даёт.
- `sandbox` — тоже не применяется в RO (у нас не используется).
- Остальные (`script-src`, `style-src`, `frame-ancestors`, ...) — **вычисляются**, нарушения попадают в отчёт, но загрузка не блокируется.

Это нормально и ожидаемо — просто помните, что «зелёный» Report-Only ≠ дословно то, что будет в enforce для этих двух директив.

10.3. Зачем вообще фаза Report-Only

Чтобы собрать список всего, что боевая политика сломала бы, **не ломая прод**. Включаешь RO → ходишь по всем сценариям → собираешь нарушения → правишь политику до нуля → и только потом флипаешь на боевой. Без этой фазы «ложно-строгая» директива положит фронт у живых пользователей.

Часть 11. Практические плейбуки

11.1. Запустить и проверить локально

```
# 1) Django (отдельный терминал)
cd journal_django
.venv/Scripts/python.exe manage.py runserver 8000
```

```
# 2) nginx перед ним (раздаёт статику + проксирует /api + принимает /csp-report)
./deploy/nginx/local/start-local-nginx.ps1
# проверить только синтаксис: ./deploy/nginx/local/start-local-nginx.ps1 -Test
# перечитать после правок: ./deploy/nginx/local/start-local-nginx.ps1 -Reload
# остановить: ./deploy/nginx/local/start-local-nginx.ps1 -Stop
```

Открыть <http://localhost:8080/>, держать открытой консоль DevTools (вкладка Console). Нарушения CSP видны там сразу. Сервер-лог: `<nginx-prefix>/logs/csp-violations.log`.

11.2. Проверить, что заголовок реально отдаётся

```
curl -I http://localhost:8080/teacher | grep -i content-security
# ожидаем строку Content-Security-Policy-Report-Only: default-src 'self'; ...
```

11.3. Разобрать лог нарушений (jq-рецепты)

```
LOG=/var/log/nginx/csp-violations.log      # локально: <nginx-prefix>/logs/csp-
violations.log

# Все нарушения «по-человечески»: какая директива + что заблокировано
jq -r '.report|fromjson|. "csp-report"|"\(.[ "effective-directive"])" ← \(.[
["blocked-uri"])"' "$LOG"

# Топ заблокированных URI
jq -r '.report|fromjson|. "csp-report"|.[ "blocked-uri"]' "$LOG" | sort | uniq -c |
sort -rn

# ОТФИЛЬТРОВАТЬ шум расширений браузера (это НЕ наши баги)
jq 'select(.report|fromjson|. "csp-report"|.[ "blocked-uri"]
| (startswith("chrome-extension")
or startswith("moz-extension")
or startswith("safari-extension"))) | not)' "$LOG"
```

11.4. Я вижу нарушение — как чинить (алгоритм)

1. Прочитай `effective-directive` (что нарушено) и `blocked-uri` (кто).
2. Реши: это **наш легитимный ресурс** или **мусор/атака**?
 - **Наш** (например, забыли вынести inline-скрипт, или добавили новый CDN) → либо **убрать inline** (предпочтительно для скриптов), либо **добавить источник** в нужную директиву (Часть 11.5).
 - **Расширение браузера** (`chrome-extension://...`) → игнор, это не мы.
 - **Подозрительный внешний URL** → не добавлять! Это, возможно, и есть атака, которую CSP поймал. Разобраться, откуда он взялся.
3. Если правил политику — обновь **оба** конфига (прод + локальный), перечитай nginx, повтори прогон.

11.5. Worked example: добавить новый разрешённый источник

Допустим, подключаем виджет аналитики с `https://cdn.metrics.example`, который грузит скрипт и шлёт данные. Нужно:

- скрипт → `script-src 'self' https://cdn.metrics.example`
- его сетевые запросы → `connect-src 'self' https://cdn.metrics.example`

Шаги:

1. В **обоих** nginx-конфигах в строке `add_header Content-Security-Policy...` добавить хост в `script-src` и `connect-src`.
2. `start-local-nginx.ps1 -Reload`, прогнать сценарий, убедиться в нуле нарушений в RO.
3. Выкатить на прод (`nginx -t && systemctl reload nginx`).

Правило: **никогда** не добавляй `'unsafe-inline'/'unsafe-eval'` в `script-src` ради стороннего скрипта. Если виджет требует inline — это красный флаг, ищи `<script src>`-версию.

11.6. Worked example (backlog): self-host Google Fonts → ужесточить политику

1. Скачать `woff2` нужных начертаний (Manrope/Inter/JetBrains Mono), положить в `/fonts/`.
2. В `teacher/index.html` и `admin-dist/index.html` заменить `<link href="https://fonts.googleapis.com/...">` на локальный `@font-face` (как уже сделано для шрифта Steppe в `teacher/styles.css`).
3. Убрать из политики `https://fonts.googleapis.com` и `https://fonts.gstatic.com`: `→ style-src 'self' 'unsafe-inline'; font-src 'self'`.
4. Прогнать RO, проверить, что шрифты грузятся и нет нарушений `font-src/style-src`.

Часть 12. CSP в связке с другими security-заголовками

CSP — не одинокий. У нас в `nginx` уже стоят (и наследуются вместе с ним):

Заголовок	Роль	Связь с CSP
<code>X-Content-Type-Options: nosniff</code>	Запрет MIME-sniffing (браузер не «угадывает» тип)	Дополняет: мешает выдать данные за скрипт/стиль
<code>X-Frame-Options: SAMEORIGIN</code>	Анти-clickjacking (legacy)	Дублируется <code>frame-ancestors</code> ; при наличии CSP современные браузеры берут <code>frame-ancestors</code>
<code>Referrer-Policy: no-referrer</code>	Не слать Referer наружу	Ортогонально
<code>Cross-Origin-Opener-Policy: same-origin</code>	Изолирует window от чужих popups	Часть cross-origin изоляции
<code>Cross-Origin-Resource-Policy: same-origin</code>	Кто может встраивать наши ресурсы	Ортогонально

Заголовок	Роль	Связь с CSP
<code>Strict-Transport-Security</code> (HSTS, эмитит Django)	Только HTTPS	<code>upgrade-insecure-requests</code> в CSP — про подресурсы, HSTS — про навигацию
<code>X-XSS-Protection: 0</code>	Выключает старый «XSS-аудитор» браузера (он сам был дырявым)	CSP — современная замена этому аудитору

Важно: `frame-ancestors 'none'` (CSP) и `X-Frame-Options: SAMEORIGIN` дают **разный** смысл («никто» против «свой origin можно»). Расхождение осознанное — `same-origin iframe` у нас нет, эффективно действует более строгое `'none'`. При появлении `iframe` (BBB-видео в кабинетах) → `frame-ancestors 'self'`.

Часть 13. Решения, которые НЕ меняем, и backlog

Сознательно оставляем

- `CSRF_COOKIE_HTTPONLY=False` — требование `double-submit`, не баг (Часть 3).
- `JWT-cookie HttpOnly=True` — не трогать (защищает саму сессию от XSS).
- `frame-ancestors 'none'` при `X-Frame-Options: SAMEORIGIN` — см. Часть 12.

Backlog (не блокирует включение CSP)

1. **Self-host Google Fonts** → `style-src 'self' 'unsafe-inline'; font-src 'self'` (плюс приватность: Google не получает IP пользователей; плюс скорость). Шаги — 11.6.
2. **Вынести inline-стили** `teacher/login` в `.css` → убрать `'unsafe-inline'` из `style-src`.
3. **report-to/Reporting-API** рядом с `report-uri` (или вместо).
4. **Обернуть `app.js` в IIFE** или `type="module"` — убрать загрязнение глобальной области (функции/`state` живут в `window`). Поведенчески-

нейтрально, но отдельной проверкой, чтобы не смешивать с CSP-рефактором.

5. При появлении **user-uploads** на нашем origin — пересмотреть `script-src 'self'`.

Часть 14. FAQ и типичные ошибки

«Включил CSP, а всё сломалось» — почти всегда забыли вынести inline-скрипт/стиль или добавить нужный источник. Поэтому и существует Report-Only: сначала RO, собрать нарушения, починить, только потом боевой.

«В логе/консоли куча `chrome-extension://` — это баги?» — Нет. Это скрипты расширений пользователя, инжектящиеся в страницу. Не наши, фильтруются (11.3).

«Отчёты идут, но в Django их нет.» — Так и задумано: `/csp-report` обрабатывает nginx (сток → 204), в Django запрос не заходит. Смотри `csp-violations.log` (Часть 8).

«`upgrade-insecure-requests` в политике есть, а http не апгрейдится локально.» — В Report-Only эта директива не действует (Часть 10.2). Заработает после флипа на боевой.

«Можно добавить `'unsafe-inline'` в `script-src`, чтобы быстро починить?» — Нет. Это обнуляет всю защиту от XSS. Чини причину (вынеси inline-JS).

«`$request_body` в логе пустой.» — Тело уехало во временный файл (отчёт > 64k) или location не проксирует. У нас `client_body_buffer_size = client_max_body_size = 64k`, а отчёты — единицы КБ, так что в норме не пусто. Если правил конфиг — проверь, что есть `proxy_pass` (без него `$request_body` всегда пуст).

«Добавил `add_header` в location — пропал CSP.» — Это правило наследования nginx (Часть 9). Либо не добавляй `add_header` в location, либо продублируй там все заголовки.

«Нужен ли CSP на JSON-ответах `/api`?» — Вреда нет, заголовок наследуется и туда. Защищает он именно HTML-страницы; на API безвреден.

Часть 15. Глоссарий

- **Origin (источник)** — `схема://хост:порт`. Единица изоляции в браузере.
- **Same-Origin Policy (SOP)** — правило: код одного origin не читает данные другого.
- **XSS** — внедрение и исполнение чужого JS внутри нашей страницы.
- **CSRF** — подделка запроса от имени жертвы с чужого сайта.
- **CSP** — заголовок, ограничивающий источники ресурсов и исполнение скриптов.
- **Директива** — правило CSP для типа ресурса (`script-src`, `style-src`, ...).
- **Список источников** — что разрешено директиве (`'self'`, хост, `data:`, `nonce`, `hash`...).
- `'self'` — тот же origin (НЕ включает `data:blob:/`поддомены).
- `'unsafe-inline'` — разрешить inline-код. В `script-src` — отключает защиту от XSS.
- `nonce` — одноразовый токен на запрос, помечающий доверенный inline-тег.
- `hash` — разрешение конкретного inline-блока по SHA его содержимого.
- **Report-Only** — режим CSP «логировать, не блокировать».
- **enforce** — боевой CSP «блокировать и логировать».
- **report-uri / report-to** — куда браузер шлёт отчёты о нарушениях.
- **Делегирование событий** — один слушатель на родителе ловит события детей через `event.target.closest(...)`.
- **Self-proxy / сток** — приём: проксировать запрос на локальный по-ор сервер, чтобы nginx прочитал тело (для логирования `$request_body`).
- **Clickjacking** — встраивание нашей страницы в чужой iframe для обмана кликами; ловится `frame-ancestors/X-Frame-Options`.

Часть 16. Шпаргалка + где это в коде

Шпаргалка

- **Зачем:** csrf читается из JS (`CSRF_COOKIE_HTTPONLY=False`) → XSS обходит CSRF → CSP не даёт исполнить чужой скрипт. Defense-in-depth.

- **Ядро:** `script-src 'self'` без `'unsafe-inline'` — `inline-JS` и `eval` запрещены.
- **Блокер был** teacher SPA → вынесли JS в `app.js`, обработчики на `addEventListener/` делегирование. `login` и `admin` были чисты.
- **Где живёт:** `nginx`, `server`-уровень, оба конфига. НЕ Django (статика без рендера → `nonce` нигде).
- **Отчёты:** браузер → `/csp-report` → `nginx self-proxy` на сток `127.0.0.1:19876` → тело в `csp-violations.log` (Django не задет).
- **Сейчас Report-Only.** Флип на боевой — после прогона 3 фронтов до нуля нарушений.
- **Грабли:** `add_header` в любом `location` убивает `server`-уровневые заголовки в нём.
- **Не лечи `'unsafe-inline'`ом** — это снимает защиту. Чини причину или добавь источник.

Где это в коде

Что	Файл
CSP-заголовков (прод) + <code>log_format csp_report</code> + сток + <code>/csp-report</code> + <code>rate-limit</code>	<code>deploy/nginx/journal-kotokod.conf</code>
CSP-заголовков (локально) + сток + <code>/csp-report</code> (без <code>rate-limit</code>)	<code>deploy/nginx/local/nginx.conf</code>
Раздача статика (наследует CSP; без своих <code>add_header</code>)	<code>deploy/nginx/snippets/journal-static.conf</code>
teacher SPA: вынесенный JS + <code>wireStaticHandlers()</code> (стр. ~135–185)	<code>journal_django/frontend/teacher/app.js</code>
teacher SPA: разметка без <code>inline on*=</code> , подключение <code>app.js</code>	<code>journal_django/frontend/teacher/index.html</code>

Что	Файл
Запуск/тест локального nginx	deploy/nginx/local/start-local-nginx.ps1
Исходный план мини-фазы + статус	journal_django/docs/security-csp-plan.md
Модель угроз cookie/JWT/CSRF (companion, планируется)	journal_django/docs/auth-explained.md
Лог нарушений: прод /var/log/nginx/csp-violations.log, локально <nginx-prefix>/logs/csp-violations.log. Разбор — Часть 11.3. Бэкап пред-рефакторной teacher/index.html (без git): journal_django/.csp-refactor-bak/.	